

Lab 11: Breaking Protections

In this lab, you will break the protections put in place by your instructor and other groups from Lab 10.

Purpose

To learn how to break protections on computer algorithms, typically those used in verifying purchases and implementing copy protections.

Procedure

You will need access to the following materials:

- A computer with a working, licensed copy of Windows or Linux
- A suite of reverse-engineering tools likely consisting of at least one static and one dynamic analysis tool.
- A software development environment for the C/C++ programming language.
- A copy of this lab's license key checker.

You have been provided with a hardened version of the license key checker from Lab 11. It has also been modified so that it implements a variant of the original algorithm, so you will need to reverse it. This will, of course, require you to circumvent the anti-reversing techniques it has implemented to protect that algorithm.

The submissions from other groups' Lab 11 will be released as they're submitted, unless something prevents me from doing so. You'll actually be seeing them for the first time just shortly after I. You'll be required to crack the instructor's checker and, optionally, another group's checker.

1. Follow the instructions given for each checker.
2. Typically, this will include an example of an incorrect and a correct username/license key combination.
3. Depending on the nature of the protections, a correct example may not be provided, but an incorrect one should be.
4. Run the checker with the provided examples to verify that it responds as expected.
5. Look at the walkthroughs in Chapter 11 and/or use online resources to begin tackling the checker.
6. The instructor-provided checker is meant to be keygen'd, not patched.
7. Example combinations:
 - DG 0123456789abcdef0123456789abcdef – Bad
 - DG b1a466bf82c19bb910eaeba9e0000090 – Good
8. Your goal is to create a program that takes as input a username and produces any valid key for that username.

Tips and Caveats

- The instructor-provided example applies two techniques, both of which are found in the textbook author's *Defender.EXE*.
- Your colleagues may have been more imaginative, but each submission should have been limited to two techniques.
- You never quite know what you're going to find in a good crackme, but with practice, you'll start to get a feel for what people tend to do, and probably hatch a few new ideas yourself.

Report

Completion of this lab must be substantiated by a report prepared by the group. It is up to each group to divide its responsibilities fairly. The report should provide sufficient detail that another person could repeat your exact or similar experiments.

Please include an archive of any source code and resulting binaries, if applicable, in your electronic submission.

Grading

The report should describe the purpose of the lab, the procedure, any assumptions, problems, or hypotheses, and the final results. Mostly, I'll be looking for a proper problem statement, a structured procedure, and for a reasonably correct solution demonstrated by your results.

This lab is due midnight after 2 weeks.

Rubric

points	Accomplishment
3	Report is well-written (organized, detailed, illustrative)
2	Report documents each protection mechanism encountered
2	Report describes how each mechanism was circumvented
1	Report describes the key checking algorithm
2	Provided source generates a valid key, given an input username, for the key checker
2	Group documents and provides a generator for another group's Lab 11 submission